

# INFORME FINAL CURSO DATA ENGINEERING

## Introducción

El presente proyecto tiene como objetivo integrar distintas fuentes de datos (archivos JSON y API REST) dentro de un pipeline completo en Azure, aplicando el modelo Medallion Architecture (Bronze – Silver – Gold).

El análisis se centra en los vuelos que conectan Buenos Aires (AEP) con Jujuy (JUJ), Salta (SLA) y Tucumán (TUC) durante la temporada alta diciembre 2024 – abril 2025, identificando cómo influyen los feriados nacionales y los días de la semana en el desempeño operativo de las aerolíneas.

## Descripción del caso de estudio:

***Análisis de estacionalidad y desempeño operativo de vuelos en el norte argentino durante la temporada alta 2024–2025***

## Descripción del problema:

Durante los meses comprendidos entre diciembre de 2024 y abril de 2025, el norte argentino experimenta un aumento sostenido de la actividad turística, especialmente en las rutas aéreas que conectan Buenos Aires (AEP) con Jujuy (JUJ), Salta (SLA) y Tucumán (TUC).

Este incremento de demanda puede afectar la regularidad de los vuelos, generando picos de cancelaciones o demoras que impactan tanto en los pasajeros como en la planificación operativa de las aerolíneas y la capacidad aeroportuaria.

El presente trabajo tiene como objetivo diseñar y construir una base analítica en Azure que integre, limpie y prepare los datos de vuelos y feriados correspondientes al período diciembre 2024 – abril 2025, permitiendo analizar posteriormente la estacionalidad y el desempeño operativo en dichas rutas.

A partir de esta integración, se busca proveer un conjunto de datos confiable y estructurado que sirva como insumo para identificar patrones de microestacionalidad, analizar la relación entre feriados, días de la semana y desempeño de aerolíneas (Flybondi, Aerolíneas Argentinas y JetSmart), y facilitar el desarrollo de estrategias de planificación o mejora del servicio aéreo durante temporadas de alta demanda.

# Orígenes y destinos de los datos

## Fuentes de datos

- Dataset principal: vuelos comerciales (JSON) con información de salidas, llegadas, cancelaciones y aerolínea, correspondientes al período 23/12/2024–17/04/2025.
  - Fuente: API de Aeropuertos Argentina / recopilación de datos históricos.
- Dataset secundario: feriados nacionales de Argentina (API pública: <https://api.argentinadatos.com/v1/feriados/2025>).

## Destino

- Azure Data Lake Storage Gen2:
  - bronze - datos crudos
  - silver - datos transformados
  - gold- datos analíticos agregados por feriados, días y aerolíneas

# Arquitectura y pipeline de datos

El proceso comienza con la ingesta de datos desde fuentes externas (APIs REST y archivos JSON), continúa con la limpieza y normalización en Databricks, y finaliza con la creación de datasets analíticos listos para consumo o visualización.

El pipeline maestro en ADF coordina todas estas etapas de forma secuencial:

Execute Feriados → Execute Bronze → Execute Silver → Execute Gold

## Infraestructura y servicios utilizados

| Servicio                            | Rol dentro del pipeline    | Descripción                                                                                                     |
|-------------------------------------|----------------------------|-----------------------------------------------------------------------------------------------------------------|
| Azure Data Factory                  | Orquestación               | Controla el flujo de ejecución completo, gestiona la ingesta desde APIs REST y ejecuta notebooks en Databricks. |
| Azure Databricks                    | Transformación             | Ejecuta notebooks para procesar, limpiar y analizar los datos con PySpark.                                      |
| Azure Data Lake Storage Gen2 (ADLS) | Almacenamiento             | Repositorio central de datos con estructura en capas (bronze, silver y gold).                                   |
| Delta Lake                          | Persistencia transaccional | Permite mantener las tablas en formato Delta con control de versiones y eficiencia en lectura/escritura.        |

# Transformaciones realizadas (en Databricks)

El flujo de transformación de datos se diseñó siguiendo la metodología Medallion Architecture, dividiendo el procesamiento en tres capas: Bronze (ingesta cruda), Silver (limpieza y normalización) y Gold (enriquecimiento y análisis).

## Capa bronze

Procesos realizados:

- 1. Ingesta desde APIs REST y archivos JSON:
  - Feriados: llamados al endpoint `https://api.argentinadatos.com/v1/feriados/{year}` para los años 2024 y 2025.
  - Vuelos: archivos `all.json` provenientes de la API de Aeropuertos Argentina (estructura JSON anidada).
- 2. Almacenamiento crudo en ADLS:
  - Los datos se almacenan en formato JSON en rutas particionadas por año y fecha.
- 3. Control de estructura y logs:
  - Validación de existencia de los archivos (`dbutils.fs.ls()`).
  - Registro de fechas de ingesta y verificación de formato (multiline JSON).

## Entradas

bronze > raw > feriados

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 2 items

| <input type="checkbox"/> | Name             | Last modified        | Access tier | Blob type |
|--------------------------|------------------|----------------------|-------------|-----------|
| <input type="checkbox"/> | folder [...]     |                      |             |           |
| <input type="checkbox"/> | folder year=2024 | 28/10/2025, 16:16:14 |             |           |
| <input type="checkbox"/> | folder year=2025 | 28/10/2025, 16:06:56 |             |           |

bronze > vuelos

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

 Search blobs by prefix (case-sensitive)

Only show active

Showing the first 100 items

| <input type="checkbox"/> | Name                                                                                                              | Last modified        | Access tier |
|--------------------------|-------------------------------------------------------------------------------------------------------------------|----------------------|-------------|
| <input type="checkbox"/> |  <a href="#">[.]</a>             |                      |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a>   | 27/10/2025, 21:30:10 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a>   | 27/10/2025, 21:30:10 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a>   | 27/10/2025, 21:30:10 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a>   | 27/10/2025, 21:30:10 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a>   | 27/10/2025, 21:30:10 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a>   | 27/10/2025, 21:30:10 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a> | 27/10/2025, 21:30:11 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a> | 27/10/2025, 21:30:11 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a> | 27/10/2025, 21:30:11 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a> | 27/10/2025, 21:30:11 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a> | 27/10/2025, 21:30:11 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a> | 27/10/2025, 21:30:11 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a> | 27/10/2025, 21:30:11 |             |
| <input type="checkbox"/> |  <a href="#">2024-12-23...</a> | 27/10/2025, 21:30:11 |             |

## Capa Silver – Limpieza, tipado y normalización

Transformaciones sobre los datos de feriados:

4. **Lectura y conversión a formato Delta:**

- Lectura de los datos de la capa bronze
- Conversión a formato Delta para mejorar la eficiencia de lectura/escritura.

5. **Normalización de nombres de columnas:**

```
.withColumnRenamed("fecha", "holiday_date_raw")
```

```
.withColumnRenamed("nombre", "name_raw")  
.withColumnRenamed("tipo", "type_raw")
```

- Cambio los nombres originales (fecha, nombre, tipo) por versiones más descriptivas (holiday\_date\_raw, name\_raw, type\_raw).
- Esto ayuda a diferenciar entre el valor original y las columnas transformadas que se crearán después.

## 6. Tipado y limpieza de valores:

```
.withColumn("holiday_date", F.to_date("holiday_date_raw", "yyyy-MM-dd"))  
.withColumn("name", F.trim(F.col("name_raw")))  
.withColumn("type", F.trim(F.col("type_raw")))
```

- F.to\_date convierte la fecha (string) al tipo date usando el formato yyyy-MM-dd.
- F.trim elimina espacios en blanco al inicio y final de los campos name y type.
- Así las columnas están correctamente tipadas y sin caracteres extra.

## 7. Filtrado

```
.filter(F.col("holiday_date").isNotNull() & F.col("name").isNotNull())
```

- Elimina cualquier registro con fecha o nombre nulo.
- Asegura que cada fila represente un feriado válido y con información completa.

## 8. Eliminación de duplicados

```
.dropDuplicates(["holiday_date", "name"])
```

- Si hay registros repetidos (mismo feriado con igual fecha y nombre), se conserva uno solo.

## 9. Selección final:

Se conservan únicamente las columnas finales (holiday\_date, name, type), generando una estructura uniforme y confiable.

## 10. Persistencia en formato Delta:

Los resultados se escriben en la ruta /silver/feriados\_delta/ con modo overwrite, y se registra la tabla capa\_silver.feriados en el catálogo Databricks.

## 11. Validación de carga:

Finalmente, se visualizan los primeros registros para confirmar que las fechas y los nombres se hayan normalizado correctamente.

## Transformaciones sobre los datos de vuelos:

### 1. Importación y lectura de bronze:

- Lectura recursiva de todos los vuelos desde la capa bronce.

## 2. Fecha de vuelo desde la ingesta (columna añadida en Bronze):

```
yyyymmdd = F.date_format(F.col("ingest_date"), "yyyy-MM-dd")
year_from_path = F.date_format(F.col("ingest_date"), "yyyy")
```

- a. Contexto: los horarios vinieron como "dd/MM HH:mm" sin año; por eso se toma el año de ingest\_date.

## 3. Función auxiliar para convertir strings de hora a timestamp

```
def to_ts(col):
    dd = F.regexp_extract(F.col(col), r"^(\d{2})/(\d{2})
(\d{2}):(\d{2})$", 1)
    MM = F.regexp_extract(F.col(col), r"^(\d{2})/(\d{2})
(\d{2}):(\d{2})$", 2)
    hh = F.regexp_extract(F.col(col), r"^(\d{2})/(\d{2})
(\d{2}):(\d{2})$", 3)
    mi = F.regexp_extract(F.col(col), r"^(\d{2})/(\d{2})
(\d{2}):(\d{2})$", 4)
    return F.to_timestamp(F.concat_ws(" ",
                                     F.concat_ws("-", year_from_path, MM, dd), # yyyy-MM-dd
                                     F.concat_ws(":", hh, mi) # HH:mm
                                ), "yyyy-MM-dd HH:mm")
```

- a. Extrae con regex día, mes, hora, minuto desde columnas tipo "dd/MM HH:mm" (e.g. stda, etda, atda).
- b. Reconstruye un timestamp concatenando año + mes + día + hora:minuto y castea con el formato yyyy-MM-dd HH:mm.

## 4. Construcción de timestamps claves

- a. Aplica la función a cada campo de horario:
  - i. sched\_ts: hora programada.
  - ii. est\_ts: hora estimada.
  - iii. act\_ts: hora real (puede ser nula si el vuelo no operó).

## 5. origen/destino según movimiento

```
origin = F.when(F.col("mov")=="D",
F.col("arpt")).otherwise(F.col("IATAdestorig"))
dest = F.when(F.col("mov")=="D",
F.col("IATAdestorig")).otherwise(F.col("arpt"))
```

- a. Esto normaliza origen/destino aun cuando la API alterna el sentido según arribo/salida.

## 6. Normalización de aerolínea

```
air_norm_map = {
```

```

"AEROLÍNEAS ARGENTINAS":"Aerolíneas Argentinas",
"AR":"Aerolíneas Argentinas",
"JETSMART":"JetSmart",
"FLYBONDI":"Flybondi"
}
from pyspark.sql.functions import create_map, lit
air_mapping = create_map([lit(x) for kv in air_norm_map.items() for x in
kv])

```

- a. Define un diccionario de mapeo para unificar variantes (siglas, mayúsculas).
- b. create\_map lo vuelve una expresión de mapa usable en Spark.

## 7. Transformación Silver completa

```

v_silver = (v
    .withColumn("flight_date", F.to_date(yyyyymmdd)) # yyyy-MM-dd
    .withColumn("sched_ts", sched_ts)
    .withColumn("est_ts", est_ts)
    .withColumn("act_ts", act_ts)
    .withColumn("origin", origin)
    .withColumn("destination", dest)
    .withColumn("airline",
F.coalesce(air_mapping[F.upper(F.col("aerolinea"))],
F.col("aerolinea")))
    .withColumn("airline_code", F.col("idaerolinea"))
    .withColumn("flight_number", F.col("nro"))
    .withColumn("delay_minutes",
        F.when(F.col("act_ts").isNotNull(),
            (F.col("act_ts").cast("long") -
F.col("sched_ts").cast("long"))/60.0)
    )
    .filter(F.length("origin")==3)
    .filter(F.length("destination")==3)
    .filter(F.col("flight_date").isNotNull())
    .filter(F.col("airline").isNotNull())
    .select(
        "flight_date","sched_ts","est_ts","act_ts",
        "origin","destination",
        "airline","airline_code","flight_number",
        "mov","estes","estin","estbr",
        "delay_minutes"
    )
)

```

- a. Nuevas columnas:
  - i. flight\_date: fecha del vuelo (desde ingest\_date).
  - ii. sched\_ts, est\_ts, act\_ts: timestamps ya convertidos.

- iii. origin, destination: normalización por mov.
  - iv. airline: nombre normalizado (usa COALESCE con mapeo o valor original).
  - v. airline\_code: código de aerolínea (idaerolinea).
  - vi. flight\_number: número de vuelo (nro).
  - vii. delay\_minutes: diferencia en minutos (solo si existe act\_ts).
  - b. Controles de calidad:
    - i. IATA de 3 caracteres para origin y destination.
    - ii. flight\_date no nula.
    - iii. airline no nula.
  - c. Selección final: deja solo las columnas analíticamente relevantes (incluye mov, estes, estin, estbr para estado/razones).
- 8. Cálculo de demoras**
- a. Se creó la métrica delay\_minutes como la diferencia, en minutos, entre el horario real (act\_ts) y el programado (sched\_ts).
  - b. Solo se calcula cuando existe act\_ts (vuelo efectivamente operado).
- 9. Controles de calidad**
- a. Se impusieron filtros para asegurar la integridad de la tabla final:
  - b. Códigos IATA de 3 caracteres en origin y destination.
  - c. flight\_date y airline no nulos.
  - d. Selección de columnas relevantes para análisis (incluyendo campos de estado mov, estes, estin, estbr).
- 10. Persistencia en Delta y registro de tabla**
- a. El resultado se guardó en /silver/vuelos\_delta y se registró como capa\_silver.vuelos en el catálogo de Databricks, lista para ser consumida por la capa Gold.

## Capa Gold

### Transformaciones sobre los datos de feriados:

1. Carga de las tablas limpias de silver:
2. Definición del período de analisis

```
start_date = "2024-12-01"
end_date   = "2025-04-30"
```

- Define el rango temporal del caso de estudio (temporada alta 2024–2025).
- Solo se consideran los vuelos ocurridos entre diciembre de 2024 y abril de 2025.

3. Filtrado de rutas de interés

```
destinos = ["JUZ", "SLA", "TUC"]
filtro_rutas = (
  ((F.col("origin")=="AEP") & (F.col("destination").isin(destinos))) |
```



```
((F.col("destination")== "AEP") & (F.col("origin").isin(destinos)))
)
```

- Crea un filtro lógico que selecciona únicamente los vuelos:
- Que salen desde AEP hacia Jujuy (JUJ), Salta (SLA) o Tucumán (TUC), o
- Que vuelven a AEP desde esas provincias.

#### 4. Aplicación del filtro temporal y de rutas

```
v_filtrado = (
    v.filter((F.col("flight_date") >= F.lit(start_date)) &
    (F.col("flight_date") <= F.lit(end_date)))
    .filter(filtro_rutas)
)
```

- Limita los vuelos al período definido.
- Aplica el filtro de rutas (AEP↔JUJ/SLA/TUC).
- El resultado (v\_filtrado) es una submuestra depurada lista para enriquecer con feriados.

#### 5. Unión (join) con la tabla de feriados

```
vj = (v_filtrado.alias("v")
    .join(h.alias("h"),
    F.col("v.flight_date")==F.col("h.holiday_date"), "left"))
```

- Hace un join por fecha entre los vuelos y los feriados.
- Se usa left join para conservar todos los vuelos (aunque no haya feriado ese día).
- Si hay coincidencia, se agregan las columnas de feriado (name, type).

#### 6. Creación de columnas derivadas

```
vj = (vj
    .withColumn("is_holiday",
    F.when(F.col("h.holiday_date").isNotNull(),
    F.lit(1)).otherwise(F.lit(0)))
    .withColumn("weekday_num", F.dayofweek("v.flight_date"))
    .withColumn("weekday_name", F.date_format("v.flight_date", "E"))
    .withColumn("delay_minutes", F.when(F.col("v.delay_minutes") >
    24*60, F.lit(None)).otherwise(F.col("v.delay_minutes"))))
```

- is\_holiday: marca los vuelos que ocurrieron en un feriado (1 = sí, 0 = no).
- weekday\_num y weekday\_name: obtienen el número y nombre del día de la semana.
- delay\_minutes: corrige valores atípicos (>24h de demora), reemplazándolos por null para evitar sesgos en el promedio.

#### 7. Detección de vuelos cancelados

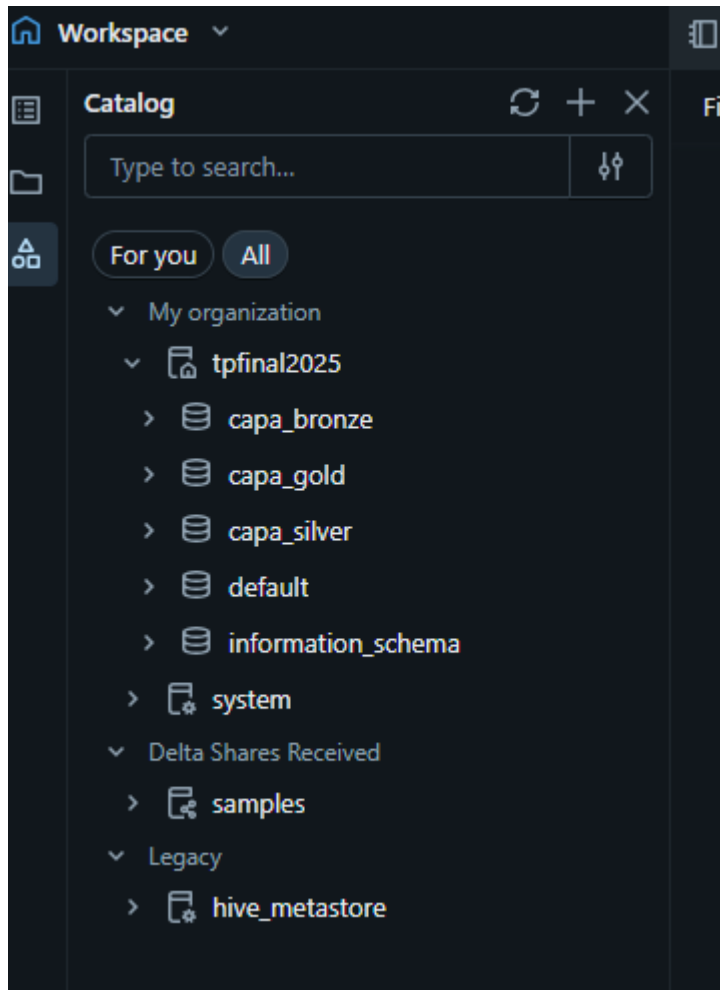
```
.withColumn("cancel_flag",
  F.when(F.col("v.act_ts").isNull(), 1)
  .when(F.upper(F.col("v.estes")).like("%CANC%"), 1)
  .when(F.upper(F.col("v.estin")).like("%CANC%"), 1)
  .when(F.upper(F.col("v.estbr")).like("%CANC%"), 1)
  .otherwise(0)
)
```

- Crea una bandera (cancel\_flag) que identifica vuelos cancelados según distintos criterios:
- No tienen hora real (act\_ts nula).
- Alguno de los campos de estado (estes, estin, estbr) contiene el texto "CANC" (cancelado).
- Esto permite calcular luego la métrica cancel\_rate (porcentaje de vuelos cancelados).

## 8. Selección de columnas finales

```
.select(
  F.col("v.flight_date"), F.col("v.sched_ts"), F.col("v.est_ts"),
  F.col("v.act_ts"),
  F.col("v.origin"), F.col("v.destination"),
  F.col("v.airline"), F.col("v.airline_code"),
  F.col("v.flight_number"),
  F.col("v.mov"), F.col("v.estes"), F.col("v.estin"),
  F.col("v.estbr"),
  "delay_minutes", "cancel_flag", "is_holiday", "weekday_num",
  "weekday_name",
  F.col("h.name").alias("holiday_name"),
  F.col("h.type").alias("holiday_type")
)
```

- Selecciona y ordena las columnas clave:
  - Datos del vuelo (fechas, aerolínea, origen, destino, estado).
  - Variables derivadas (delay\_minutes, cancel\_flag, is\_holiday).
  - Enriquecimiento de contexto (holiday\_name, holiday\_type).
- El resultado (vj) es una tabla enriquecida donde cada vuelo incluye información sobre:
  - su demora,
  - su condición de cancelado o no,
  - el día de la semana,
  - y si ocurrió o no en feriado.



## Controles de calidad implementados

Durante el desarrollo del pipeline se implementaron controles de calidad en cada capa del modelo Medallion para garantizar la consistencia, completitud y fiabilidad de los datos procesados.

En la capa Bronze, se verificó la estructura de los archivos JSON antes de su carga, asegurando que no existieran archivos vacíos o con formato incorrecto (por ejemplo, exclusión de all-keys.json).

En la capa Silver, se aplicaron validaciones de tipado y contenido: se eliminaron registros con valores nulos en campos críticos (flight\_date, origin, destination, airline, holiday\_date), se filtraron códigos IATA inválidos y se deduplicaron filas repetidas tanto en vuelos como en feriados. Además, se validaron los rangos de las demoras, descartando valores negativos o mayores a 24 horas.

Finalmente, en la capa Gold, se controló la consistencia de los joins (para evitar duplicidad de registros por fecha), y se verificó que las métricas agregadas (delay\_mean, cancel\_rate) se calcularan solo sobre vuelos válidos.

Estos mecanismos garantizan que los resultados finales sean coherentes, reproducibles y listos para su análisis o visualización en herramientas BI.

# Automatización

El pipeline se ejecuta con trigger manual (modo prueba), con actividades secuenciales: Copy Feriados, Copy Vuelos, Databricks Silver, Databricks Gold. Puede configurarse un trigger diario para automatizar el flujo.

## Resultados y análisis

La capa Gold consolidó la información de vuelos y feriados en un dataset analítico que permite estudiar patrones de desempeño operativo por ruta, día y aerolínea.

A partir de los campos `delay_mean`, `cancel_rate`, `is_holiday` y `weekday_name`, el conjunto de datos quedó preparado para que los analistas puedan evaluar el impacto de los feriados, la distribución de demoras por día de la semana y las diferencias operativas entre aerolíneas.

## Conclusiones y reflexiones finales

Durante el desarrollo se abordaron desafíos técnicos relevantes, como:

- La heterogeneidad de los archivos JSON de vuelos (distintas estructuras y nombres de campos).
- La configuración de Linked Services y datasets en Azure Data Factory, especialmente al definir rutas dinámicas y parámetros.
- La necesidad de manejar grandes volúmenes de archivos en el contenedor Bronze, optimizando la lectura con PySpark.
- La transformación de fechas sin año y la construcción de campos derivados (`delay_minutes`, `cancel_flag`, `is_holiday`) para garantizar coherencia temporal.
- A pesar de estas dificultades, el pipeline logró ejecutarse correctamente, entregando datos limpios, enriquecidos y con calidad controlada.
- Este resultado demuestra la capacidad de integrar Azure Data Factory, Data Lake y Databricks de manera orquestada y escalable.

Sin embargo, se reconocen algunas limitaciones del dataset:

- El análisis cubre solo cuatro meses, por lo que no se incluyen comportamientos anuales o históricos.
- Los datos provienen de archivos JSON heterogéneos, lo que requirió una limpieza exhaustiva.
- La API de origen no ofrece métricas meteorológicas ni de ocupación, que podrían enriquecer futuros estudios.

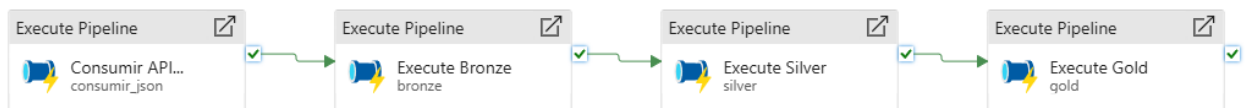
En términos técnicos, los principales desafíos estuvieron relacionados con el volumen de archivos JSON, la configuración de Linked Services y datasets en ADF, y el manejo de transformaciones con PySpark en Databricks.

# Referencias

- API Aeropuertos Argentina (Datos históricos de vuelos obtenidos a partir de fuentes públicas y registros en línea, complementados con información de failbondi.fail - sitio que documenta la disponibilidad y estado de vuelos comerciales en Argentina).
- API ArgentinaDatos (feriados) <https://api.argentinadatos.com/v1/feriados>
- Documentación oficial de Azure: ADF, ADLS Gen2, Databricks
- Notebooks y scripts propios del proyecto

## Capturas:

### Pipeline final



### Ejecuciones

Pipeline runs

Triggered

Debug

Rerun

Cancel options

Refresh

Edit columns

List

Gantt

Filter by run ID or name

Brasilia (UTC-3) : Last 24 hours

Pipeline name : All

Status : All

Runs : Latest runs

Triggered by : All

Add filter

Copy filters

Export to CSV

Showing 1 - 11 items

Last refreshed 0 minutes ago

| <input type="checkbox"/> | Pipeline name ↑↓ | Run start ↑↓           | Run end ↑↓             | Duration | Triggered by          | Status ↑↓ | Run            | Parameters | Annotations | Run ID                               |
|--------------------------|------------------|------------------------|------------------------|----------|-----------------------|-----------|----------------|------------|-------------|--------------------------------------|
| <input type="checkbox"/> | gold             | 10/30/2025, 7:04:56 PM | 10/30/2025, 7:05:36 PM | 40s      | 696ed8bd-31e8-4aa...  | Succeeded | Original       |            |             | 13bf77a6-c212-4f1f-a543-a772aac5b78f |
| <input type="checkbox"/> | silver           | 10/30/2025, 7:04:19 PM | 10/30/2025, 7:04:55 PM | 37s      | 1c35b57d-6746-4df...  | Succeeded | Original       |            |             | 2d825e40-0c01-4ab8-b76a-053584f69f9f |
| <input type="checkbox"/> | bronze           | 10/30/2025, 7:01:10 PM | 10/30/2025, 7:04:17 PM | 3m 8s    | 5e8c9154-8925-406c... | Succeeded | Original       |            |             | 1a79181c-321d-407a-a953-69b9ef6db55f |
| <input type="checkbox"/> | consumir_json    | 10/30/2025, 7:00:41 PM | 10/30/2025, 7:01:08 PM | 28s      | d0dadc9f-fa5a-47f2... | Succeeded | Original       |            |             | 87d013a1-cb9d-4f8d-8366-31db0c2041f1 |
| <input type="checkbox"/> | master           | 10/30/2025, 7:00:32 PM | 10/30/2025, 7:05:38 PM | 5m 6s    | Manual trigger        | Succeeded | Original       |            |             | 0a6c4056-5f6f-4aae-9ba0-939ada4c932f |
| <input type="checkbox"/> | gold             | 10/30/2025, 6:38:36 PM | 10/30/2025, 6:38:59 PM | 24s      | 8b1365a6-6c50-434...  | Succeeded | Original       |            |             | 10e2cd71-2fc5-4135-9f7b-977aebbb92f1 |
| <input type="checkbox"/> | silver           | 10/30/2025, 6:37:54 PM | 10/30/2025, 6:38:34 PM | 41s      | 8f48cfce-6f3d-4a3e... | Succeeded | Original       |            |             | d0ebfcfc-2181-44bf-a4eb-1a83bbf40b2f |
| <input type="checkbox"/> | > bronze         | 10/30/2025, 6:34:55 PM | 10/30/2025, 6:37:52 PM | 2m 58s   | 9e79f19a-d29b-4f9b... | Succeeded | Rerun (Latest) |            |             | 00f15c68-d1c1-4010-8499-368288ec88b8 |
| <input type="checkbox"/> | > master         | 10/30/2025, 6:34:49 PM | 10/30/2025, 6:39:00 PM | 4m 12s   | Manual trigger        | Succeeded | Rerun (Latest) |            |             | 51fc2322-a229-44a5-ae6b-e8525e9e6e0f |
| <input type="checkbox"/> | bronze           | 10/30/2025, 6:05:35 PM | 10/30/2025, 6:08:43 PM | 3m 8s    | e55c372a-0f95-4820... | Failed    | Original       |            |             | c363ff3b-7f0b-4e91-9cfd-09607e3a422f |
| <input type="checkbox"/> | master           | 10/30/2025, 6:05:26 PM | 10/30/2025, 6:08:45 PM | 3m 19s   | Manual trigger        | Failed    | Original       |            |             | 6fb01b23-42d7-44a0-b2c2-12874b2191f1 |

Jobs & Pipelines

Create new

Ingestion pipeline

Ingest data from popular apps, databases and file sources

ETL pipeline

Build ETL pipelines using SQL and Python

Job

Orchestrate notebooks, pipelines, queries and more

Jobs & pipelines

Job runs

Send feedback

Job

Run as

Start: 28/10/2025 20:00

End: 30/10/2025 20:00

Run status

Error code

Create job

1

1

0

29 Oct, 12 AM

29 Oct, 12 PM

30 Oct, 12 AM

30 Oct, 12 PM

Failed

Skipped

Succeeded

| Start time             | Job                                  | Run as          | Launched           | Duration | Status    | Error code        | Run parameters |  |
|------------------------|--------------------------------------|-----------------|--------------------|----------|-----------|-------------------|----------------|--|
| Oct 30, 2025, 07:05 PM | ADF_tp102025_gold_Ejecutar Gol...    | Veronica Valdez | By runs submit API | 16s      | Succeeded |                   |                |  |
| Oct 30, 2025, 07:04 PM | ADF_tp102025_silver_Ejecutar Silv... | Veronica Valdez | By runs submit API | 23s      | Succeeded |                   |                |  |
| Oct 30, 2025, 07:01 PM | ADF_tp102025_bronze_Ejecutar Br...   | Veronica Valdez | By runs submit API | 2m 37s   | Succeeded |                   |                |  |
| Oct 30, 2025, 06:38 PM | ADF_tp102025_gold_Ejecutar Gol...    | Veronica Valdez | By runs submit API | 14s      | Succeeded |                   |                |  |
| Oct 30, 2025, 06:37 PM | ADF_tp102025_silver_Ejecutar Silv... | Veronica Valdez | By runs submit API | 21s      | Succeeded |                   |                |  |
| Oct 30, 2025, 06:35 PM | ADF_tp102025_bronze_Ejecutar Br...   | Veronica Valdez | By runs submit API | 2m 34s   | Succeeded |                   |                |  |
| Oct 30, 2025, 06:06 PM | ADF_tp102025_bronze_Ejecutar Br...   | Veronica Valdez | By runs submit API | 2m 3s    | Failed    | RunExecutionError |                |  |
| Oct 30, 2025, 06:05 PM | ADF_tp102025_bronze_Ejecutar Br...   | Veronica Valdez | By runs submit API | 2m 36s   | Failed    | RunExecutionError |                |  |